NETWORKWALKS

Wireshark SOC Labs Complete Progress Report

Consolidated record of practical learning, packet-analysis concepts, answers discussed, and completion status

Student	Vansh Saxena
Track	Cybersecurity Internship – Week 1 / Wireshark practicals
Environment	VirtualBox lab environment; Kali Linux / Windows; Wireshark
Report date	5 November 2026
Coverage	Prior Networkwalks conversations through the current chat

Important accuracy note: This report records only details available in the conversation history. Where a packet capture, screenshot, exact answer, or task result was not preserved in chat, it is marked **not recorded / needs verification** rather than guessed.

1. Executive summary

The work focused on practical Wireshark analysis for a SOC / Blue Team learning path: identifying suspicious network activity, following TCP streams, inspecting HTTP requests, investigating login attempts, and analysing VoIP signalling and voice traffic.

The conversation history records early TCP and HTTP exercises, an FTP login-attack investigation, a brute-force lab marked complete by the student, and a VoIP call-analysis lab marked complete by the student. The latest conversation specifically covered SIP, RTP, SIP INVITE fields, RTP payload type, sensitive information spoken over a call, and encryption controls.

Completion snapshot

Area / lab	Status in chat	Evidence retained
TCP SYN scan / attacker-IP identification	Previously completed exercise	Exact attacker IP not recorded
TCP Follow Stream – office chat	Completed	TCP/5000; password and blue text recorded
Follow TCP Stream – Part 2	100/100 reported	/staff.html 200 OK; /download/report.txt
HTTP plain-text credentials	Completed in learning sequence; details partial	POST known; exact path not recorded
FTP login investigation	Student confirmed complete	Exact server IP, username, password not preserved
Brute-force investigation / Lab 7 sequence	Student confirmed Lab 7 complete	Exact title/findings not fully preserved
VoIP / Lab 8 – Catch the Call Scammer	Student confirmed complete	SIP/RTP concepts recorded; exact caller/codec/card digits not recorded

Lab numbering note

The earlier course/module summary described six browser-based exercises by title, while later messages referred to FTP, Lab 7, and Lab 8. The numbering and titles are not fully reconciled in the saved chat history. This report therefore separates the known exercise names from the later student-reported completion labels instead of inventing a definitive mapping.

2. Practical work and findings

A. TCP SYN scan – identify the attacker's IP

• Learning objective: inspect a packet capture for TCP SYN scanning behaviour and identify the source IP generating the scan.

• Wireshark approach: inspect TCP conversations and packet details; look for repeated SYN packets directed at multiple ports or hosts.

• Useful filter: `tcp.flags.syn == 1 and tcp.flags.ack == 0`.

• **Recorded result:** the exercise was reported as completed, but the exact attacker IP and packet counts are not present in the retained conversation.

B. Follow TCP Stream – office chat (TCP port 5000)

• Learning objective: reconstruct application data exchanged over a TCP connection.

• Wireshark approach: identify the TCP conversation, right-click a packet, and choose **Follow** → **TCP Stream**.

• Recorded details: office-chat traffic used TCP port **5000**; the conversation history records the temporary password **Tiger#4821** and that the relevant client chat text was **blue**.

Security lesson: unencrypted application traffic can expose credentials and private messages to anyone able to capture the traffic. Use TLS and avoid transmitting passwords in plaintext.

C. Follow TCP Stream – Part 2

• Completion score recorded in chat: **100/100** on 5 November 2026.

• Recorded web findings: `/staff.html` returned **200 OK**; a file was downloaded from `/download/report.txt`.

• The hidden-page answer and other task-specific values were not retained in the conversation.

D. HTTP plain-text credentials / sign-in form

• The lab question asked which HTTP method the sign-in form used and which page path received the submitted values.

• Recorded answer: HTTP method was **POST**.

• The exact request path was not preserved in the chat. It must be read from the HTTP request line / packet details rather than guessed.

• Useful filter: `http.request`. Inspect the packet's Hypertext Transfer Protocol section for Request Method and Request URI.

3. FTP and login-attack investigation

FTP login investigation

The student worked through questions about an attacked FTP server, repeated username attempts, and the password used in a successful login, then confirmed that the lab was complete.

Wireshark workflow used / recommended

Display filter for FTP username requests:

```
ftp.request.command == "USER"
```

Display filter for FTP password requests:

```
ftp.request.command == "PASS"
```

Look for the server reply **230** (successful login) and correlate it with the preceding USER and PASS commands in the same TCP stream. FTP without TLS sends credentials in plaintext.

Exact findings not recorded: the FTP server IP address, repeatedly attempted username, and successful password were not supplied in a screenshot or written answer in the retained chat. They should not be reconstructed from guesses.

Brute-force investigation / Lab 7

• The student explicitly stated that Lab 7 was complete.

• The exact task title, target service, attacker IP, account name, and number of failed attempts are not fully preserved.

• General investigation workflow: identify repeated authentication failures, group by source IP and username, establish the time window, and check whether a successful login followed the failures.

• Useful starting filters depend on the protocol: SSH commonly uses `ssh`; FTP uses `ftp`; HTTP authentication should be inspected with `http.request`.

SOC investigation checklist

• What is the source IP and destination IP?

• Which service / destination port is targeted?

• Which username or account is targeted?

• How many failures occurred, and over what time period?

• Was there a successful login after the failures?

• What evidence supports the conclusion? Record packet numbers and timestamps.

4. Lab 8 – VoIP Call Replay / Catch the Call Scammer

The student completed the Networkwalks Lab 8 VoIP exercise. The official page shared in chat was networkwalks.com/wireshark-lab8/. The conversation covered the following questions and concepts.

SIP – call setup and signalling

• **Answer:** SIP (Session Initiation Protocol) sets up and manages the call.

• **INVITE:** caller initiates the call.

• **180 Ringing:** recipient's phone is ringing.

• **200 OK:** recipient accepts / call is answered.

• Useful display filter: `sip`. To isolate INVITE messages: `sip.Method == "INVITE"`.

RTP – voice media

• **Answer:** RTP (Real-time Transport Protocol) carries the actual voice/audio packets.

• Useful display filter: `rtp`.

• The lab described many small voice packets arriving about every 60 ms.

Finding the extension and caller name

• Select the SIP INVITE packet and expand **Session Initiation Protocol**.

• Inspect the **From** header. The SIP URI commonly contains the caller's extension; the display name may appear before the URI.

• **Exact extension and caller name:** not recorded in the chat because the INVITE packet details were not shared.

Finding the voice codec

• Select an RTP packet and inspect its **Payload type**. Use the SIP/SDP session description to map dynamic payload types (often 96–127) to codecs.

• Common static payload types: 0 = PCMU (G.711 μ-law); 8 = PCMA (G.711 A-law); 9 = G.722; 18 = G.729.

• **Exact codec:** not recorded; no actual payload-type value was provided.

Recovering spoken information

• The question asked for the last four digits of a card number spoken during the call.

• The recommended analysis path was Telephony → RTP → Streams, select the stream, analyse / play or export the audio, and listen to the relevant portion.

• **Exact last four digits:** not recorded in chat. Do not infer or invent payment-card digits.

Protecting calls from eavesdropping

• **SRTP** encrypts RTP media (voice packets).

• **SIP over TLS** protects SIP signalling.

• Use strong key management, network segmentation, access controls, and secure VoIP configuration. A VPN can provide additional protection for remote connections but does not replace properly configured SRTP/TLS.

Key distinction: SIP controls the call; RTP transports the voice; SRTP protects the voice media.

5. Wireshark command and filter quick reference

Purpose	Display filter / menu path
Show TCP traffic	`tcp`
TCP SYN without ACK (common scan indicator)	`tcp.flags.syn == 1 and tcp.flags.ack == 0`
HTTP requests	`http.request`
FTP traffic	`ftp`
FTP USER commands	`ftp.request.command == "USER"`
FTP PASS commands	`ftp.request.command == "PASS"`
SIP signalling	`sip`
SIP INVITE	`sip.Method == "INVITE"`
RTP voice packets	`rtp`
Protocol breakdown	Statistics → Protocol Hierarchy
Endpoint / conversation overview	Statistics → Endpoints / Conversations
Reconstruct TCP application data	Right-click packet → Follow → TCP Stream
Inspect RTP calls	Telephony → RTP → Streams